

IBM vs IonQ

초전도 vs 이온 트랩

2026.07.28

2025113574 권나현

System Software Lab.

목차

1. 주제 선정 이유
2. 현재 양자 하드웨어 생태계
3. 실습: QPU Simulator의 QAOA 구현
4. 실습: IBM QPU의 QAOA 구현
5. 실습: IonQ QPU의 QAOA 구현
6. 실습: 결과 시각화, 성능 비교하기
7. 결론

1. 주제 선정 이유

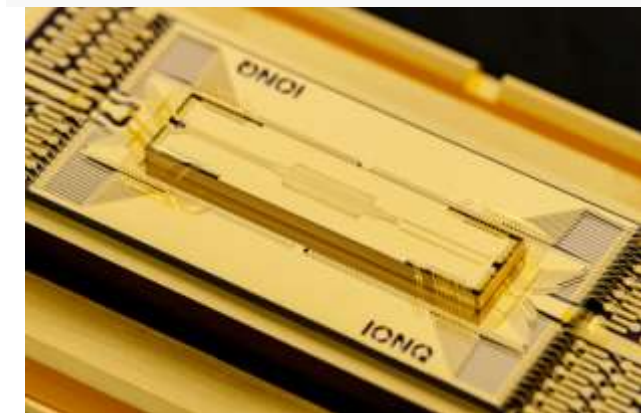
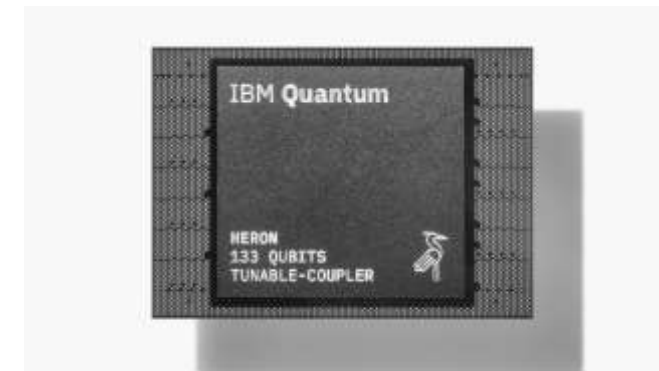
- 양자 컴퓨팅 생태계의 다양화: 다양한 양자 하드웨어 공정 기술의 공존
- 초전도와 이온트랩 하드웨어의 성능 차이가 알고리즘 결과에 미치는 영향을 알아보기 위함
- Max-cut QAOA 구현
- 알고리즘 결과의 관점에서 양자 하드웨어의 성능 비교하기

2. 현재 양자 하드웨어 생태계

양자 하드웨어 공정 방법들

대표적인 양자컴퓨터 큐비트 구현 방식

구분	초전도 (Superconducting)	이온트랩 (Trapped Ion)	광자 (Photonic)	중성원자 (Neutral Atoms)
원리	초전도 상태의 회로에 조셉슨 접합을 이용	진공 상태에서 이온을 레이저로 포획·조작	빛의 편광이나 위상 차이를 이용	중성원자를 레이저로 포획·조작
특징	가장 보편적 방식, 대형화 가능	낮은 오류, 안정성	높은 균일성, 확장성, 상온 작동(가능성)	빠른 속도, 상온 작동, 기존 반도체 공정 활용
발전단계	높음	높음	실험적 단계	실험적 단계
운영비	매우 높음 (극저온 냉각 유지)	높음 (진공 및 레이저 유지)	비교적 낮음	비교적 낮음
글로벌 대표기업	(미국) IBM, 구글, 리게티컴퓨팅 (중국) 바이두, 오리진퀀텀 IBM Google rigetti Baidu 百度 本源量子 IONQ QUANTINUUM	(미국) 아이온큐, 켄티넘 IONQ QUANTINUUM	(캐나다) 자나두 (미국) PSI퀀텀 XANADU PsiQuantum	(프랑스) 파스칼 (미국) 큐에라 PASQAL IQEura Computing Inc.



3. 실습: QPU의 QAOA 구현

⋮ QAOA 실습 규모

파라미터 최적화 루프: 1000 shots

최적화된 파라미터 적용: 10000 shots

(시뮬레이터, IBM과 IONQ 모두 동일)

최적화된 파라미터 적용 시 사용한 각도(모두 동일)

`optimal_params = [2.86932684, 1.47976977, 3.4937362, 3.3565659]`

3. 실습: QPU Simulator의 QAOA 구현

최적화된 파라미터 적용

▶ #최적의 각도를 적용한 최종 양자 회로를 실행하여 정답(최적해) 확인하기

```
sampler = Sampler(mode=backend)
sampler.options.default_shots = 10000 # 회로 루프의 반복 횟수

# 오류 제어 옵션 설정
sampler.options.dynamical_decoupling.enable = True # 동적 디커플링
sampler.options.dynamical_decoupling.sequence_type = "XY4"
sampler.options.twirling.enable_gates = True # 파울리 트윙링
sampler.options.twirling.num_randomizations = "auto"

sampler.options.environment.job_tags = ["TUT_QAOA"] # 작업 태그 지정

pub = (optimized_circuit,) # 원소 1개짜리 튜플 생성
job = sampler.run([pub], shots=int(1e4)) #pub을 양자 컴퓨터에서 10000번 실행
counts_int = job.result()[0].data.meas.get_int_counts()
counts_bin = job.result()[0].data.meas.get_counts()
shots = sum(counts_int.values())
final_distribution_int = {key: val / shots for key, val in counts_int.items()}
final_distribution_bin = {key: val / shots for key, val in counts_bin.items()}
final_distribution_bin_shots = {key: val for key, val in counts_bin.items()}
print(final_distribution_int)
print(final_distribution_bin_shots)
```

... {10: 0.0421, 11: 0.1113, 26: 0.0735, 4: 0.0038, 23: 0.008, 9: 0.107, 14: 0.0285, 5: 0.0737, 3: 0.0154, 21: 0.0408, 28: 0.0137, 20: 0.1126, 18: 0.0438, 13: 0.0401, 27: 0.0037, 6: 0.0159, '01010': 421, '01011': 1113, '11010': 735, '00100': 38, '10111': 80, '01001': 1070, '01110': 285, '00101': 737, '00011': 154, '10101': 408, '11100': 137, '10100': 1126, '10010': 438, '...

3. 실습: QPU Simulator의 QAOA 구현

그래프 시각화

```

▶ #그래프 그리기
plt.rcParams.update({"font.size": 10})
final_bits = dict(sorted(final_distribution_bin_shots.items()))
values = np.abs(list(final_bits.values()))
top_4_values = sorted(values, reverse=True)[:4]

positions = []
for value in top_4_values:
    positions.append(np.where(values == value)[0])

fig = plt.figure(figsize=(11, 6))
ax = fig.add_subplot(1, 1, 1)

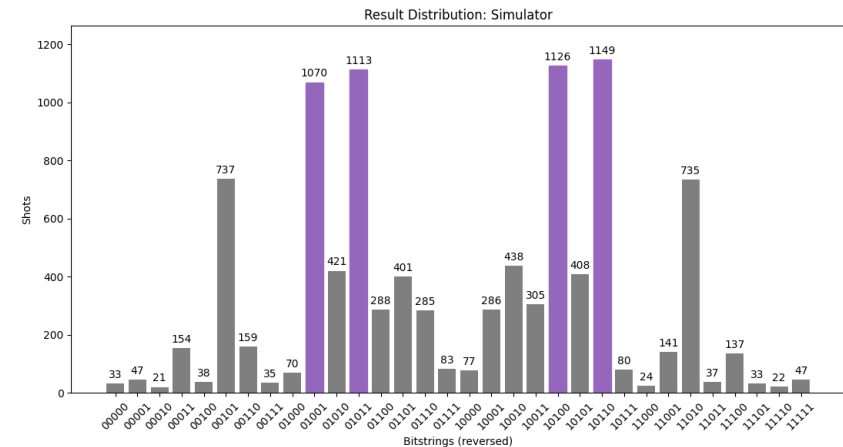
plt.xticks(rotation=45)
plt.title("Result Distribution: Simulator")
plt.xlabel("Bitstrings (reversed)")
plt.ylabel("Shots")

bars = ax.bar(list(final_bits.keys()), list(final_bits.values()), color="tab:grey")

for p in positions:
    ax.get_children()[int(p[0])].set_color("tab:purple")

ax.bar_label(bars, padding=3)
ax.set_ylim(0, max(final_bits.values()) * 1.1)
plt.tight_layout()

plt.show()
```



4. 실습: IBM QPU의 QAOA 구현

최적화된 파라미터 적용

```
#최적의 각도를 적용한 최종 양자 회로를 IBM에 제출하여 정답(최적해) 확인하기 (최초 1회)
from qiskit.compiler import transpile
from qiskit_ibm_runtime import SamplerV2, EstimatorV2

...

sampler = SamplerV2(mode=backend)
sampler.options.default_shots = 10000 # 회로 루프의 반복 횟수

# 오류 제어 옵션 설정
sampler.options.dynamical_decoupling.enable = True # 동적 디커플링
sampler.options.dynamical_decoupling.sequence_type = "XY4"
sampler.options.twirling.enable_gates = True # 파울리 트윙링
sampler.options.twirling.num_randomizations = "auto"
sampler.options.environment.job_tags = ["TUT_QAOA"] # 작업 태그 지정

transpiled_circuit = transpile(optimized_circuit, backend=backend)

job = sampler.run([transpiled_circuit])
print(f"Job ID: {job.job_id()}")
...
```

... Job ID: d9h41h4honhs73acho20

```
#이전에 IBM에 제출하여 받은 결과 불러오기
retrieved_job = service.job("d9h41h4honhs73acho20")
counts_int = retrieved_job.result()[0].data.meas.get_int_counts()
counts_bin = retrieved_job.result()[0].data.meas.get_counts()
shots = sum(counts_int.values())
final_distribution_int = {key: val / shots for key, val in counts_int.items()}
final_distribution_bin = {key: val / shots for key, val in counts_bin.items()}
final = {key: val for key, val in counts_bin.items()}
print(final_distribution_int)
print(final_distribution_bin)
print(final)
```

```
... {3: 0.0172, 13: 0.0442, 20: 0.0939, 9: 0.0948, 26: 0.0685, 11: 0.1042, 19: 0.0322, 10: 0.0479, 12: 0.0262, 21: 0.0414, 23: 0.0101, 6: 0.0154, 29: 0.0058, 22: 0.0986, 28: 0.0155, 18:
{'00011': 0.0172, '01101': 0.0442, '10100': 0.0939, '01001': 0.0948, '11010': 0.0685, '01011': 0.1042, '10011': 0.0322, '01010': 0.0479, '01100': 0.0262, '10101': 0.0414, '10111': 0.
{'00011': 172, '01101': 442, '10100': 939, '01001': 948, '11010': 685, '01011': 1042, '10011': 322, '01010': 479, '01100': 262, '10101': 414, '10111': 101, '00110': 154, '11101': 58,
```


4. 실습: IBM QPU의 QAOA 구현

최적화된 파라미터 적용

[All workloads](#) /

d9h41h4honhs73acho20

Actions



Details

[Edit tags](#)

User

Nahyeon Kwon

Region

Washington DC (us-east)

Program

sampler

Instance

[open-instance](#)

Mode

Job

of PUBs

1

Quantum computer

ibm_fez

[View calibration history](#)

Tags

TUT_QAOA

Status and usage

Status

✓ Completed

Total completion time

2m 50s

Total usage

5s

✓ Created: Jul 24, 2026, 1:26 AM

✓ Pending: 31s

✓ In progress: Jul 24, 2026, 1:27 AM

Usage: 5s

✓ Completed: Jul 24, 2026, 1:29 AM

4. 실습: IBM QPU의 QAOA 구현

그래프 시각화

```
▶ top_4_values = sorted(values, reverse=True)[:4]

positions = []
for value in top_4_values:
    positions.append(np.where(values == value)[0])

fig = plt.figure(figsize=(11, 6))
ax = fig.add_subplot(1, 1, 1)

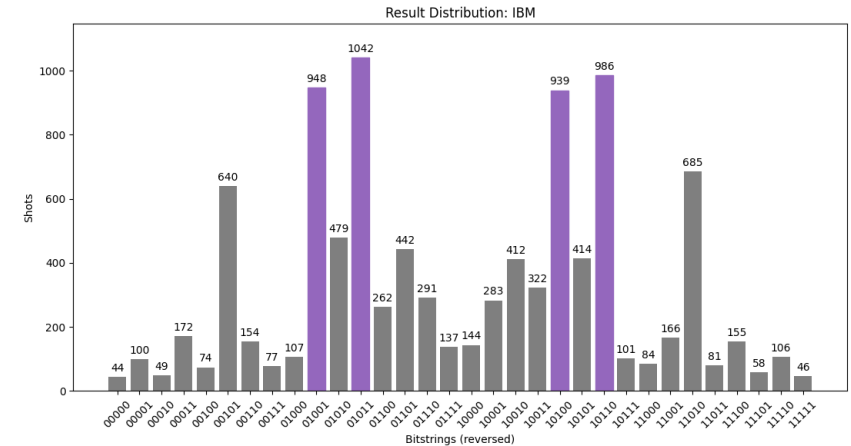
plt.xticks(rotation=45)
plt.title("Result Distribution: IBM")
plt.xlabel("Bitstrings (reversed)")
plt.ylabel("Shots")

bars = ax.bar(list(final_bits.keys()), list(final_bits.values()), color="tab:grey")

for p in positions:
    ax.get_children()[int(p[0])].set_color("tab:purple")

ax.bar_label(bars, padding=3)
ax.set_ylim(0, max(final_bits.values()) * 1.1)
plt.tight_layout()

plt.show()
```



5. 실습: IonQ QPU의 QAOA 구현

최적화된 파라미터 적용

```
from qiskit.primitives import SamplerV2, EstimatorV2

sampler = SamplerV2(mode=backend)
sampler.options.default_shots = 10000 # 회로 루프의 반복 횟수

# 오류 제어 옵션 설정
sampler.options.dynamical_decoupling.enable = True # 동적 디커플링
sampler.options.dynamical_decoupling.sequence_type = "XY4"
sampler.options.twirling.enable_gates = True # 파울리 트윙링
sampler.options.twirling.num_randomizations = "auto"

sampler.options.environment.job_tags = ["TUT_QAOA"] # 작업 태그 지

pub = optimized_circuit
job = backend.run([pub], shots=int(1e4)) #pub을 양자 컴퓨터에서 10000번 실행
```



#이전에 IonQ에 제출하여 받은 결과 불러오기

```
retrieved_job = backend.retrieve_job("019f7b09-f16c-763b-8266-bd4ee4940905")
counts_bin = retrieved_job.result().get_counts()
```



1. 도착한 결과 객체 가져오기

```
result = retrieved_job.result()
```

...

2. 이진수 결과 카운트 꺼내기 (IonQ 전용 문법)

```
counts_bin = result.get_counts()
```

```
print("최종 결과 측정 카운트(이진수):")
```

```
print(counts_bin)
```

...

```
counts_bin = result.get_counts()
```

```
counts_int = {int(k, 2): v for k, v in counts_bin.items()}
```

```
print(counts_int)
```

```
shots = sum(counts_int.values())
```

```
final_distribution_int = {key: val / shots for key, val in counts_int.items()}
```

```
final_distribution_bin = {key: val / shots for key, val in counts_bin.items()}
```

```
final_distribution_bin_shots = {key: val for key, val in counts_bin.items()}
```

```
print(final_distribution_int)
```

```
print(final_distribution_bin)
```

```
print(final_distribution_bin_shots)
```

...

```
{11: 1100, 22: 1041, 9: 1037, 20: 990, 26: 633, 5: 624, 21: 472, 13: 472, 18: 464, 10: 443, 17: 280, 12: 277, 19: 267, 14: 253, 25: 178, 3: 174, 28: 161, 6: 158, 30: 107, 27: 102, 4: 13}
{11: 0.11, 22: 0.1041, 9: 0.1037, 20: 0.099, 26: 0.0633, 5: 0.0624, 21: 0.0472, 13: 0.0472, 18: 0.0464, 10: 0.0443, 17: 0.028, 12: 0.0277, 19: 0.0267, 14: 0.0253, 25: 0.0178, 3: 0.0174,
{'01011': 0.11, '10110': 0.1041, '01001': 0.1037, '10100': 0.099, '11010': 0.0633, '00101': 0.0624, '10101': 0.0472, '01101': 0.0472, '10010': 0.0464, '01010': 0.0443, '10001': 0.028,
{'01011': 1100, '10110': 1041, '01001': 1037, '10100': 990, '11010': 633, '00101': 624, '10101': 472, '01101': 472, '10010': 464, '01010': 443, '10001': 280, '01100': 277, '10011': 267,
```

5. 실습: IonQ QPU의 QAOA 구현

최적화된 파라미터 적용

Job Name	1 circuits
Job ID	019f7b09-f16c-763b-8266-bd4ee4940905
Project Name	Personal Workspace (2025113574@kumoh.ac.kr)
Project ID	3cd0d94b-4883-40b0-b41f-9323cb873853
Status	✔ completed
Backend	forte-1
Type	circuit
Circuits	1
Dry run	False
Cost	\$168.20
Cost Model	2QGE operations
Requested	2026-07-20 00:41:17 GMT+9
Completed	2026-07-20 00:51:17 GMT+9
Executed in	00:04:08.327
Predicted	00:02:24.641

5. 실습: IonQ QPU의 QAOA 구현

그래프 시각화

```
plt.rcParams.update({"font.size": 10})
final_bits = dict(sorted(final_distribution_bin_shots.items()))
values = np.abs(list(final_bits.values()))
top_4_values = sorted(values, reverse=True)[:4]

positions = []
for value in top_4_values:
    positions.append(np.where(values == value)[0])

fig = plt.figure(figsize=(11, 6))
ax = fig.add_subplot(1, 1, 1)

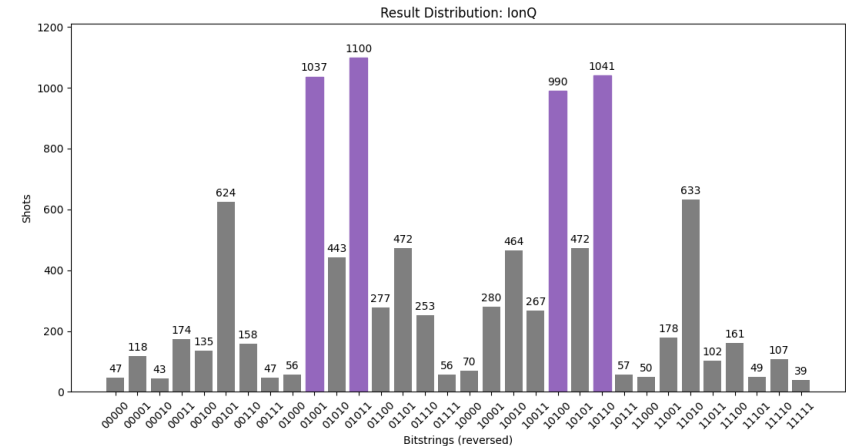
plt.xticks(rotation=45)
plt.title("Result Distribution: IonQ")
plt.xlabel("Bitstrings (reversed)")
plt.ylabel("Shots")

bars = ax.bar(list(final_bits.keys()), list(final_bits.values()), color="tab:grey")

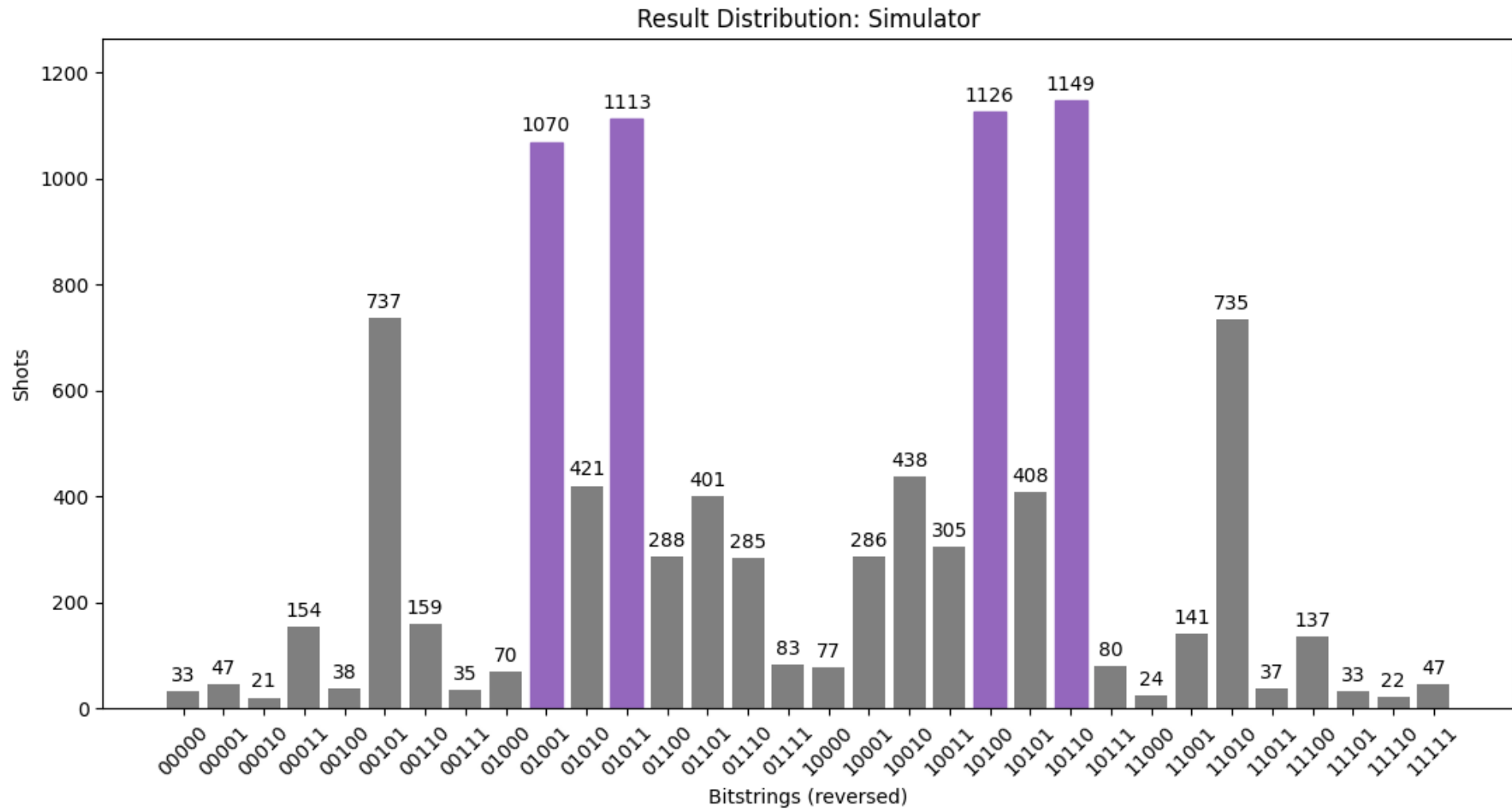
for p in positions:
    ax.get_children()[int(p[0])].set_color("tab:purple")

ax.bar_label(bars, padding=3)
ax.set_ylim(0, max(final_bits.values()) * 1.1)
plt.tight_layout()

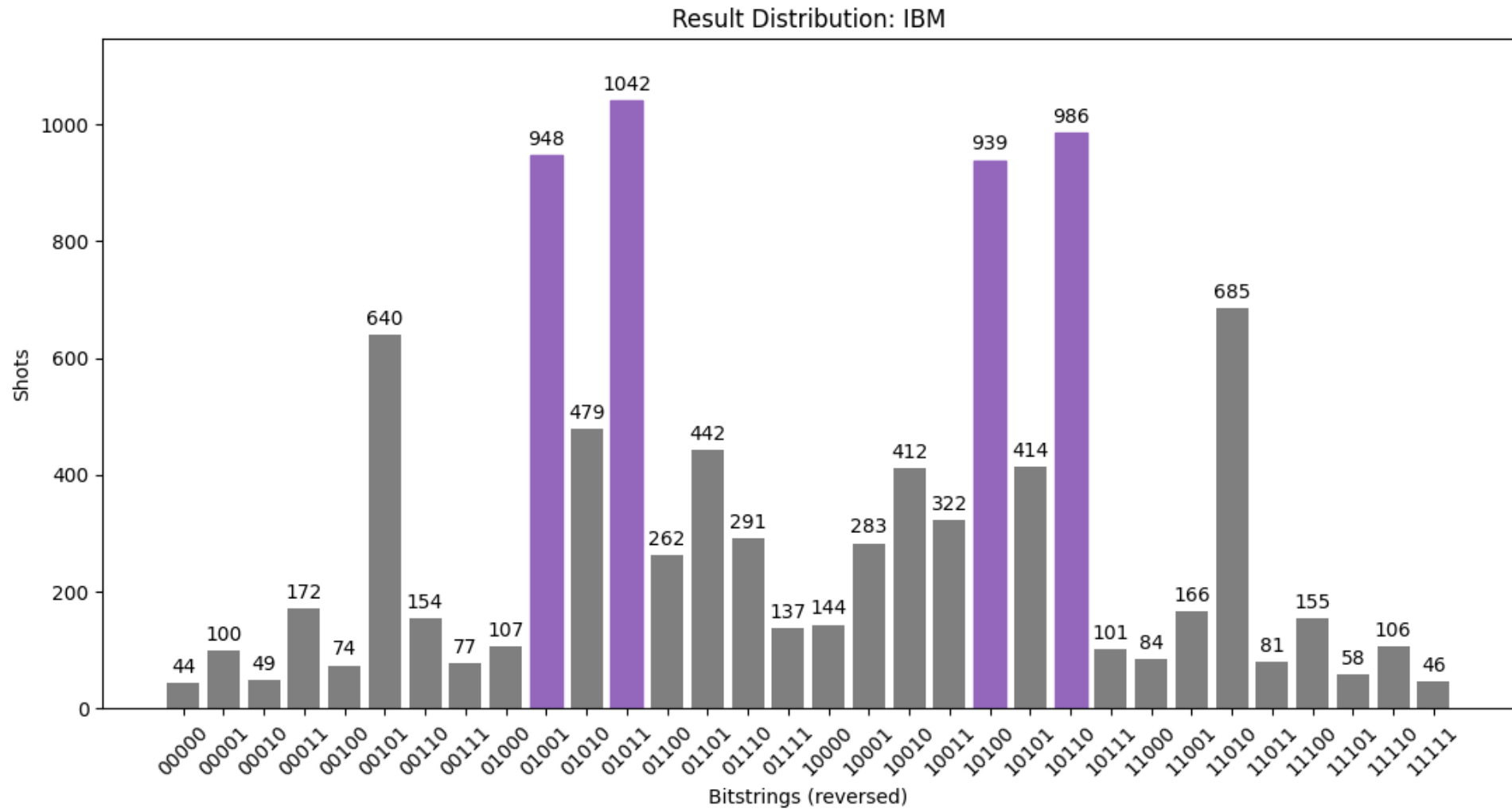
plt.show()
```



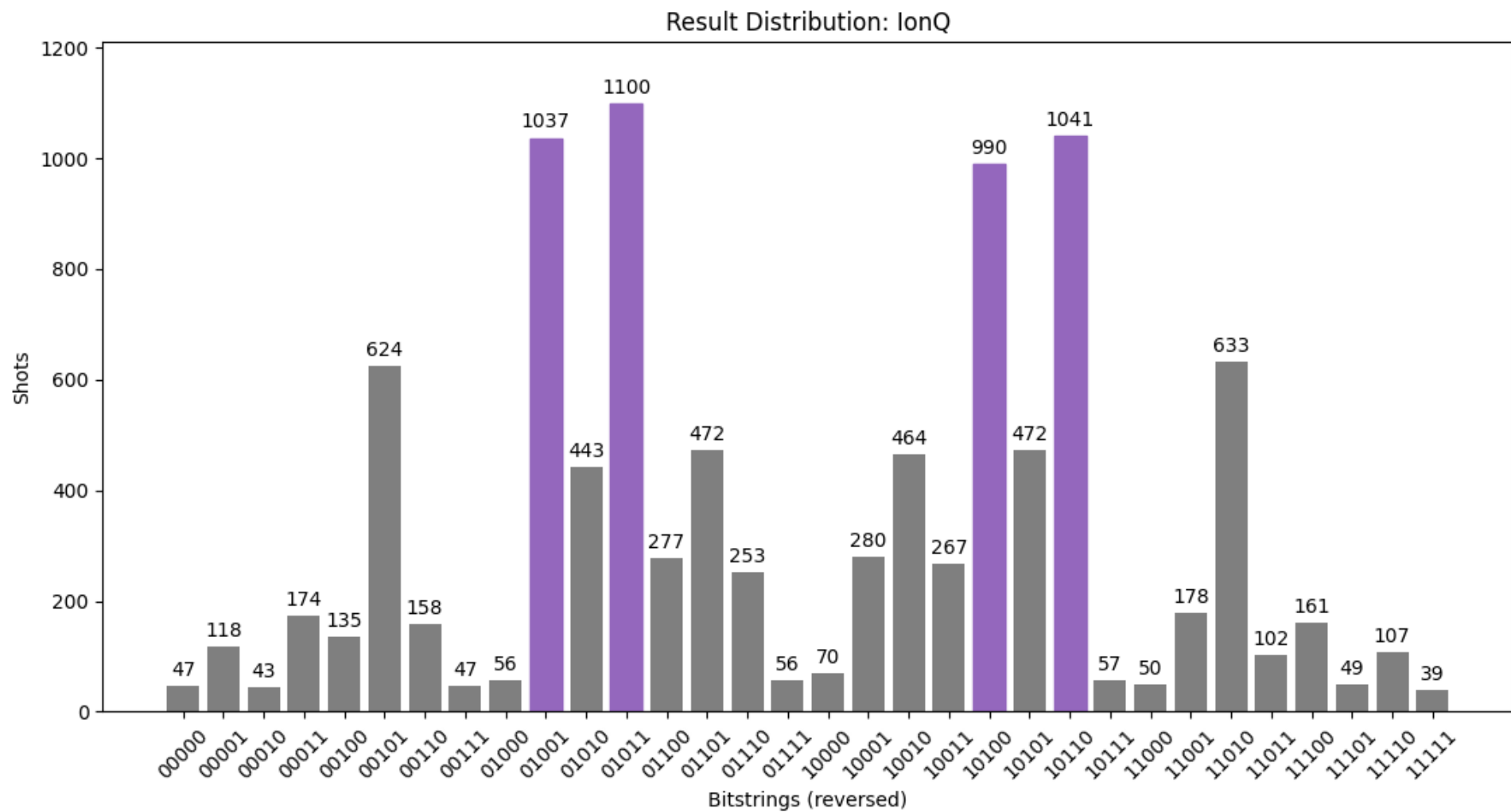
6. 결과 시각화



6. 결과 시각화



6. 결과 시각화



6. 성능 비교하기

양자 하드웨어 성능 분석 – 알고리즘 결과 관점

최적해 도출 확률 비교 (정답 측정 횟수 / 전체 샷)

[1]
✓ 0초

```
simulator = {'01010': 421, '01011': 1113, '11010': 735, '00100': 38, '10111': 80, '01001': 1070, '01110': 285, '00101': 737, '00011': 154, '10101': 408, '11100': 137, '10100': 1126, '10110': 288, '01000': 70, '00000': 33, '00001': 47, '00010': 21, '00011': 154, '00100': 38, '00101': 737, '00110': 159, '00111': 35, '01000': 70, '01001': 1070, '01010': 421, '01011': 1113, '01100': 288, '01101': 262, '01110': 277, '10010': 464, '10001': 280, '10011': 472, '10101': 472, '10110': 443, '10111': 472, '11010': 633, '11011': 1100, '11100': 277, '11101': 277, '11110': 277, '11111': 277, '10000': 47, '10001': 118, '10010': 43, '10011': 174, '10100': 135, '10101': 624, '10110': 158, '10111': 47, '11000': 56, '11001': 1037, '11010': 443, '11011': 1100, '11100': 277, '11101': 277, '11110': 277, '11111': 277}
```

[2]
✓ 0초

```
final_sim = dict(sorted(simulator.items()))  
final_ibm = dict(sorted(ibm.items()))  
final_ionq = dict(sorted(ionq.items()))
```

[3]
✓ 0초

```
print(final_sim)  
print(final_ibm)  
print(final_ionq)
```

▼

```
... {'00000': 33, '00001': 47, '00010': 21, '00011': 154, '00100': 38, '00101': 737, '00110': 159, '00111': 35, '01000': 70, '01001': 1070, '01010': 421, '01011': 1113, '01100': 288, '01101': 262, '01110': 277, '01111': 277, '10000': 47, '10001': 118, '10010': 43, '10011': 174, '10100': 135, '10101': 624, '10110': 158, '10111': 47, '11000': 56, '11001': 1037, '11010': 443, '11011': 1100, '11100': 277, '11101': 277, '11110': 277, '11111': 277}
```

6. 성능 비교하기

양자 하드웨어 성능 분석 – 알고리즘 결과 관점

최적해 도출 확률 비교 (정답 측정 횟수 / 전체 샷)

[5]
✓ 0초

```
import numpy as np

sim_values = np.abs(list(final_sim.values()))
sim_top_4 = sorted(sim_values, reverse=True)[:4]
print(sim_top_4)

ibm_values = np.abs(list(final_ibm.values()))
ibm_top_4 = sorted(ibm_values, reverse=True)[:4]
print(ibm_top_4)

ionq_values = np.abs(list(final_ionq.values()))
ionq_top_4 = sorted(ionq_values, reverse=True)[:4]
print(ionq_top_4)
```

... [np.int64(1149), np.int64(1126), np.int64(1113), np.int64(1070)]
[np.int64(1042), np.int64(986), np.int64(948), np.int64(939)]
[np.int64(1100), np.int64(1041), np.int64(1037), np.int64(990)]

[16]
✓ 0초

```
probability_sim = 0
probability_ibm = 0
probability_ionq = 0

for i in sim_top_4:
    probability_sim += i
print("Simulator의 최적해 도출 확률:", (probability_sim / 10000) * 100, "%")

for i in ibm_top_4:
    probability_ibm += i
print("IBM QPU의 최적해 도출 확률:", (probability_ibm / 10000) * 100, "%")

for i in ionq_top_4:
    probability_ionq += i
print("IonQ QPU의 최적해 도출 확률:", (probability_ionq / 10000) * 100, "%")
```

... Simulator의 최적해 도출 확률: 44.58 %
IBM QPU의 최적해 도출 확률: 39.15 %
IonQ QPU의 최적해 도출 확률: 41.68 %



```
print("IBM QPU의 상대적 유지율:", (probability_ibm / probability_sim) * 100, "%")
print("IonQ QPU의 상대적 유지율:", (probability_ionq / probability_sim) * 100, "%")
```

... IBM QPU의 상대적 유지율: 87.81965006729476 %
IonQ QPU의 상대적 유지율: 93.49484073575594 %

7. 결론

QPU의 QAOA 최적해 도출 성능

- Simulator (이상치): 44.58% (32개 경우의 수 중 무작위 탐색 12.5% 대비 3.5배 이상의 성능)
- IonQ (이온트랩): 41.68% (시뮬레이터 대비 93.5% 성능 유지)
- IBM (초전도): 39.15% (시뮬레이터 대비 87.8% 성능 유지)
- 5큐비트(소규모) : 두 QPU 모두 이상적 시뮬레이터 대비 87% 이상의 정밀도 달성
- 소규모 회로에서는 두 하드웨어 모두 우수한 성능을 보임
- 큐비트 수가 확장됨에 따른 성능 우위에 대해서는 차후 탐구할 계획

감사합니다.